

# Atividade 3: Pontos de Interesse e seus Descritores

Pipeline completo · Aplicações · Descritores Neurais

## Introdução

Nesta atividade você irá explorar o **pipeline completo** de detecção, descrição e correspondência de pontos de interesse, aplicando-o em um problema real de montagem de imagens. Ao final, você terá um primeiro contato prático com descritores modernos baseados em aprendizado de máquina. Sem entrar na matemática das redes, não abordada nos livros de graduação no caso dos algoritmos mais complexos, mas compreendendo o que eles oferecem na prática.

Estrutura da Atividade

- **Tarefa 1:** Espaço de Escala, Visualização e Análise de Descritores
- **Tarefa 2:** Descritores Float vs. Binários — Comparação Prática
- **Tarefa 3:** Aplicação — Montagem de Panorama (Image Stitching)
- **Tarefa 4:** Exploração — Descritores Neurais com Kornia (DISK)

Pipeline Geral de Keypoints

Imagem 1 e Imagem 2 → detectar keypoints → extrair descritores → calcular correspondências → filtrar matches → estimar transformação geométrica → aplicar na tarefa final.

## O que são Keypoints?

Os **pontos de interesse (keypoints)** são características únicas em uma imagem (cantos, interseções ou regiões com textura distintiva) que permitem identificar o mesmo objeto em diferentes condições (rotação, escala, iluminação). Eles são acompanhados de um **descritor**, que funciona como uma “impressão digital” visual daquela vizinhança e permite encontrar correspondências entre imagens diferentes. Vide slides e livro do Corke.

## Aplicações Práticas

- Montagem de panoramas e mosaicos de imagens (*image stitching*)
- Localização e mapeamento simultâneos (SLAM) em robótica e drones
- Realidade aumentada, rastreamento visual e registro de objetos
- Veículos autônomos, *visual odometry*
- Reconhecimento e rastreamento de objetos em vídeo
- Reconstrução 3D a partir de fotos (*Structure from Motion*)

## Material de Apoio

Consulte os **slides da aula de Pontos de Interesse (TECA2 20261)** e as referências indicadas em cada seção. Nesta atividade, você usará implementações prontas do OpenCV e da biblioteca Kornia no Google Colab. A atividade tomará **três semanas** (09/05/2026 a 30/05/2026) conforme o cronograma abaixo. Organizem o trabalho ao longo das três semanas. Dúvidas poderão ser discutidas no Lab. 100 aos sábados ou por e-mail, conforme combinado no início do semestre.

## Entrega Final

A entrega deverá conter os notebooks das quatro tarefas com o uso do OpenCV/Bibliotecas, as imagens utilizadas nos experimentos, as tabelas preenchidas e uma conclusão final curta. Os resultados devem ser reprodutíveis no Google Colab. Notebooks que apresentarem apenas código e figuras, sem comentários ou interpretação dos resultados, não receberão pontuação plena nos critérios de análise crítica e apresentação.

| Cronograma (Prazo Total: 3 Semanas — 09/05/2026 a 30/05/2026) |                                                                                                                                                                                                                                                                                                                                                                |
|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dias 1–7<br>(09/05 a 15/05)                                   | <b>Tarefas 1 e 2 — Espaço de Escala e Descritores</b> <ul style="list-style-type: none"><li>- Leitura: Corke Caps. 12–13; Gonzalez &amp; Woods, Cap. 9; Slides 8–16.</li><li>- Pirâmide Gaussiana, visualização de keypoints com DRAW_RICH_KEYPOINTS.</li><li>- Comparação float (SIFT) vs. binário (ORB, AKAZE) com Ratio Test.</li></ul>                     |
| Dias 8–14<br>(16/05 a 22/05)                                  | <b>Tarefa 3 — Image Stitching</b> <ul style="list-style-type: none"><li>- Leitura: Corke Seção 14.1–14.2.4; Tutorial OpenCV de Homografia.</li><li>- Fotografar pares de imagens com celular (sobreposição de 30–50%).</li><li>- Pipeline completo: detectar → descrever → corresponder → warp.</li></ul>                                                      |
| Dias 15–19<br>(23/05 a 27/05)                                 | <b>Tarefa 4 — Descritores Neurais</b> <ul style="list-style-type: none"><li>- Leitura: Slide 20 da aula; Torralba et al. (2024), Cap. 11; Kornia docs.</li><li>- Instalar Kornia, rodar DISK, comparar com SIFT.</li></ul>                                                                                                                                     |
| Dias 20–21<br>(28/05 a 30/05)                                 | <b>Entrega Final e Preparação para Apresentação</b> <ul style="list-style-type: none"><li>- Revisão dos notebooks, tabelas comparativas e conclusões.</li><li>- Prepare-se para apresentar presencialmente seus resultados em aula às 8h50 na Sala 201 como temos feito desde o início.</li><li>- <b>Submissão via Google Classroom: 30/05/2026.</b></li></ul> |

# Tarefa 1 — Espaço de Escala, Visualização e Análise de Descritores

### Objetivos de Aprendizagem

- Compreender pirâmide gaussiana e espaço de escala
- Visualizar keypoints com escala e orientation (DRAW\_RICH\_KEYPOINTS)
- Interpretar o que o descritor SIFT de 128 componentes captura de uma região
- Comparar detecção do SIFT e AKAZE em imagens com diferentes texturas

## 1.1 Leitura e Estudo

- Corke (2023), Cap. 12 — Seção 12.3.2, *Scale-Space Concepts*
- Gonzalez & Woods (2018), *Digital Image Processing*, 4ª Ed. — Cap. 9, Seções 9.1–9.2 (Pirâmides de Imagens, suavização gaussiana): base matemática mais detalhada sobre filtros e pirâmides do que o Corke
- Slides da aula (TECA2 20261): slides 8–12 (SIFT, espaço de escala)
- OpenCV SIFT: [https://docs.opencv.org/4.x/da/df5/tutorial\\_py\\_sift\\_intro.html](https://docs.opencv.org/4.x/da/df5/tutorial_py_sift_intro.html)
- OpenCV AKAZE: [https://docs.opencv.org/4.x/db/d70/tutorial\\_akaze\\_matching.html](https://docs.opencv.org/4.x/db/d70/tutorial_akaze_matching.html)

## 1.2 Ação — Implementação no Colab

1. **Pirâmide Gaussiana e DoG:** Construa manualmente (com OpenCV) a pirâmide gaussiana de uma imagem (pelo menos 3 oitavas) e visualize as imagens em cada nível. Em seguida, compute a Diferença de Gaussianas (DoG) entre níveis adjacentes e mostre o resultado. O objetivo é visualizar a estrutura multiescala que serve de base para a detecção de keypoints no SIFT.
2. **Visualização rica de keypoints:** Usando ao menos 3 imagens de sua escolha (prefira texturas variadas: plana, com cantos, padrões repetitivos), detecte keypoints com SIFT e AKAZE. Visualize com DRAW\_RICH\_KEYPOINTS. Observe que os círculos mostram escala e orientação de cada ponto.

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('imagem.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

sift = cv2.SIFT_create()
akaze = cv2.AKAZE_create()

kp_s, d_s = sift.detectAndCompute(gray, None)
kp_a, d_a = akaze.detectAndCompute(gray, None)

f = cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
out_s = cv2.drawKeypoints(img, kp_s, None, flags=f)
out_a = cv2.drawKeypoints(img, kp_a, None, flags=f)

print(f'SIFT : {len(kp_s)} kp | '
      f'{None if d_s is None else d_s.shape}')
print(f'AKAZE: {len(kp_a)} kp | '
      f'{None if d_a is None else d_a.shape}')
```

3. **Análise do descritor:** Para um keypoint selecionado, imprima os 128 valores do vetor SIFT e visualize-os como um gráfico de barras (plt.bar(range(128), d\_s[0])). Responda: o que esses valores representam? Por que 128 dimensões?
4. **Efeito da escala:** Redimensione uma imagem para 50% e 25% do tamanho original. Detecte keypoints com SIFT nas três versões. Os mesmos pontos são encontrados nas três escalas? Discuta o que isso revela sobre a invariância a escala do SIFT.

### Dica Prática

- Use imagens com ao menos 400×400 px.
- Redimensionar: cv2.resize(img, None, fx=0.5, fy=0.5)
- Gráfico de barras do descritor: plt.bar(range(128), desc[i])

## 1.3 Critérios de Avaliação (25% cada)

1. **Leitura e Compreensão:** Clareza na explicação do espaço de escala e DoG. Referências citadas.
  2. **Implementação:** Pirâmide visualizada; keypoints detectados e exibidos corretamente.
  3. **Análise Crítica:** Interpretação do descritor SIFT de 128 componentes; discussão sobre invariância a escala.
  4. **Apresentação:** Notebook organizado com figuras, legendas e conclusões claras.
- Submissão:** Google Classroom — Tarefa1.ipynb e Tarefa1.pptx.

# Tarefa 2 — Descritores Float vs. Binários: Comparação Prática

### Objetivos de Aprendizagem

- Compreender a diferença entre descritores float (SIFT) e binários (ORB, AKAZE)
- Entender quando usar NORM\_L2 vs. NORM\_HAMMING no matching
- Aplicar o Lowe’s Ratio Test e analisar seu efeito na qualidade dos matches
- Comparar visualmente o matching com cada tipo de descritor

## 2.1 Leitura e Estudo

- Slides da aula: slides 12–16 (SIFT vs. ORB, Matching, BF, Ratio Test)
- Corke (2023), Seção 13.3 — Feature Matching
- Lowe (2004), Seção 7.1 — Ratio Test (artigo original do SIFT)
- OpenCV Matching: [https://docs.opencv.org/4.x/dc/dc3/tutorial\\_py\\_matcher.html](https://docs.opencv.org/4.x/dc/dc3/tutorial_py_matcher.html)

### Conceito-chave: por que a métrica importa?

Descritores **float** (SIFT, 128 floats) medem magnitudes contínuas — use a distância euclidiana **NORM\_L2**. Descritores **binários** (ORB, AKAZE) armazenam bits comparando pares de pixels; use a **distância de Hamming** (**NORM\_HAMMING**) — muito mais rápida de calcular.

## 2.2 Ação — Implementação no Colab

1. **Par de imagens:** Use a mesma cena fotografada de dois ângulos levemente diferentes, ou uma imagem e sua versão rotacionada 20–30°. Este par será seu banco de testes para toda a Tarefa 2.
2. **Matching com cada descritor:** Execute o pipeline completo para SIFT, ORB e AKAZE:

```
import cv2

def pipeline_match(img1, img2,
                  det_name='SIFT', ratio=0.75):
    g1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
    g2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
    if det_name == 'SIFT':
        det = cv2.SIFT_create()
        norm = cv2.NORM_L2
    elif det_name == 'ORB':
        det = cv2.ORB_create(nfeatures=2000)
        norm = cv2.NORM_HAMMING
    elif det_name == 'AKAZE':
        det = cv2.AKAZE_create()
        norm = cv2.NORM_HAMMING
    else:
        raise ValueError('det_name invalido')
    kp1, d1 = det.detectAndCompute(g1, None)
    kp2, d2 = det.detectAndCompute(g2, None)
    if d1 is None or d2 is None:
        print(f'{det_name}: descritores insuficientes')
        return kp1, kp2, []
    bf = cv2.BFMatcher(norm)
    pairs = bf.knnMatch(d1, d2, k=2)
    good = []
    for pair in pairs:
        if len(pair) == 2:
            m, n = pair
            if m.distance < ratio * n.distance:
                good.append(m)
    n_ok = len(good)
    n_tot = len(pairs)
```

```
print(f'{det_name}: {len(kp1)} kp, '
      f'{n_ok}/{n_tot} bons')
return kp1, kp2, good
```

3. **Efeito do Ratio Test:** Para o SIFT, varie o limiar entre 0.50, 0.65, 0.75 e 0.90. Registre a quantidade de matches aceitos em cada caso e visualize com `cv2.drawMatches()`. Analise o trade-off quantidade  $\times$  qualidade.
4. **Tabela comparativa:** Preencha a tabela abaixo com seus resultados reais:

| Método | Distância      | KPs | Match (%) |
|--------|----------------|-----|-----------|
| SIFT   | L2 (float)     | —   | —         |
| ORB    | Hamming (bin.) | —   | —         |
| AKAZE  | Hamming (bin.) | —   | —         |

Match (%) = matches bons ÷ total de pares candidatos.

Dica: visualizando os matches

- Use `cv2.drawMatches(img1, kp1, img2, kp2, good, None, flags=2)`.
- Matches ruins formam linhas cruzadas e caóticas — compare antes/depois do Ratio Test.
- ORB pode gerar mais falsos matches em imagens com texturas repetitivas.

### 2.3 Critérios de Avaliação (25% cada)

1. **Fundamentos:** Explica corretamente a diferença float/binário e as métricas de distância.
  2. **Implementação:** Código correto para os 3 detectores, BFMatcher configurado adequadamente.
  3. **Análise Crítica:** Tabela preenchida; efeito do Ratio Test discutido com evidências.
  4. **Apresentação:** Figuras com matches visualizados; notebook claro e bem comentado.
- Submissão: Google Classroom — Tarefa2.ipynb e Tarefa2.pptx.

## 3 Tarefa 3 — Aplicação: Montagem de Panorama (Image Stitching)

Objetivos de Aprendizagem

- Aplicar o pipeline completo em uma tarefa real de montagem
- Estimar a homografia entre duas imagens com RANSAC
- Aplicar warp perspectivo e fundir duas imagens em um panorama
- Avaliar a qualidade do resultado em função do detector e do número de inliers

### 3.1 Leitura e Estudo

- Corke (2023), Seções 14.1–14.2.4 — *Point Feature Correspondence* e *Planar Homography*
- Slides da aula: slides 15–18 (Matching, Stitching, RANSAC)
- OpenCV Tutorial: [https://docs.opencv.org/4.x/d7/dff/tutorial\\_feature\\_homography.html](https://docs.opencv.org/4.x/d7/dff/tutorial_feature_homography.html)
- Brown & Lowe (2007), *AutoStitch* — artigo de referência para stitching com keypoints

O que é uma Homografia?

Uma **homografia** é uma transformação geométrica que relaciona dois planos de imagem, preservando linhas retas. Para montar um panorama, usamos a homografia para reprojeter (*warp*) uma das imagens no sistema de coordenadas da outra e depois as fundimos. Leia Corke, pág. 604.

**RANSAC** estima a homografia usando apenas as correspondências mais consistentes, descartando automaticamente os *outliers* tal como expliquei na aula. Outliers são matches incorretos que atrapalhariam o cálculo. O Corke fala do RANSAC na pág. 601, leia

com atenção e também o destaque no Excuse 14.3 da pág. 602.

Como fotografar para o stitching

- Gire a câmera aproximadamente em torno do mesmo ponto, evitando deslocar lateralmente o celular entre as fotos.
- Garanta sobreposição de **30–50%** entre as duas imagens.
- Evite cenas com muito movimento (pessoas, folhas ao vento).
- Boas opções: corredores, fachadas, estantes de livros, paredes com cartazes.

### 3.2 Ação — Implementação no Colab

1. **Coleta de imagens:** Use seu celular para fotografar ao menos 2 pares de imagens com sobreposição adequada. As imagens originais usadas nos experimentos devem ser entregues junto com o notebook ou incorporadas ao ambiente do Colab.
2. **Pipeline de stitching:** Implemente as etapas abaixo. Use SIFT para a versão principal:

```
import cv2, numpy as np

img1 = cv2.imread('esquerda.jpg')
img2 = cv2.imread('direita.jpg')
g1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
g2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

# 1. Detectar e descrever
sift = cv2.SIFT_create()
kp1, d1 = sift.detectAndCompute(g1, None)
kp2, d2 = sift.detectAndCompute(g2, None)
if d1 is None or d2 is None:
    raise RuntimeError('Descritores insuficientes')

# 2. Matching com Ratio Test
bf = cv2.BFMatcher(cv2.NORM_L2)
pairs = bf.knnMatch(d1, d2, k=2)
good = []
for pair in pairs:
    if len(pair) == 2:
        m, n = pair
        if m.distance < 0.75 * n.distance:
            good.append(m)
print(f'Bons matches: {len(good)}')

# 3. Homografia com RANSAC
if len(good) >= 10:
    src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
    dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
    M, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
    inliers = int(mask.ravel().sum())
    print(f'Inliers: {inliers}/{len(good)} '
          f'({100*inliers/len(good):.1f}%)')

# 4. Warp e composicao simples
h1, w1 = img1.shape[:2]
h2, w2 = img2.shape[:2]
result = cv2.warpPerspective(img1, M, (w1 + w2, max(h1, h2)))
result[0:h2, 0:w2] = img2
else:
    print('Poucos matches para estimar homografia')
```

**Dica — estimador moderno:** em versões recentes do OpenCV ( $\geq 4.5$ ), troque `cv2.RANSAC` por `cv2.USAC_MAGSAC` dentro de `cv2.findHomography()` para usar o USAC/MAGSAC, mais rápido e mais robusto a outliers que o RANSAC clássico. O restante do código permanece idêntico. Em pares fáceis o ganho costuma ser pequeno; em cenas com muitos falsos matches, é hoje a escolha padrão. Outras opções da família: `cv2.USAC_DEFAULT`, `cv2.USAC_ACCURATE`.

**Observação:** esta composição é propositalmente simples e serve para fins didáticos. Ela não corrige translação negativa (quando a homografia mapeia `img1` para coordenadas  $x < 0$  ou  $y < 0$ ), não ajusta automaticamente o canvas a partir do *bounding box* das imagens warpadas e não faz *blending* na região de sobreposição. Caso o panorama fique cortado, deslocado ou com a costura visível, registre o problema no notebook e discuta a causa — a versão “correta” calcularia o canvas final a partir dos cantos transformados e aplicaria uma translação para acomodar tudo.



3. **Comparação SIFT × ORB:** Repita o pipeline com ORB no mesmo par. Registre keypoints detectados, matches bons, taxa de inliers e qualidade visual do panorama.
4. **Discussão:** O panorama ficou bom com ambos? Onde há diferença visível? Em que situação você preferiria ORB em vez de SIFT em um sistema real?

Como saber se o stitching está correto?

Sinais de sucesso:

- Região de sobreposição alinhada sem dupla imagem.
- Linhas retas da cena continuam retas no panorama.
- Taxa de inliers > 60%.

Sinais de falha:

- Objetos aparecem duplicados ou com halo.
- Bordas muito distorcidas na região de junção.
- Taxa de inliers < 30%.

**Se falhar com poucos matches** (<10 bons), aumente o limiar de Lowe (0.75 → 0.85) e o threshold do RANSAC (5.0 → 8.0): você fica mais permissivo, ganhando matches.

**Se falhar com matches ruidosos** (inliers < 30%), reduza o limiar de Lowe (0.75 → 0.65) e o threshold do RANSAC (5.0 → 3.0): você fica mais rigoroso, descartando outliers.

Tabela Comparativa

| Detector | KP (img1) | Matches | Inliers (%) | OK? |
|----------|-----------|---------|-------------|-----|
| SIFT     | —         | —       | —           | —   |
| ORB      | —         | —       | —           | —   |

3.3 Critérios de Avaliação (25% cada)

1. **Fundamentação:** Explica claramente homografia, RANSAC e seu papel no stitching.
2. **Implementação:** Código funcional com warp e composição; panorama gerado e exibido.
3. **Análise Comparativa:** Tabela preenchida; diferenças SIFT × ORB discutidas com evidências.
4. **Apresentação:** Panoramas exibidos com zoom na região de junção; conclusões fundamentadas.
- Submissão: Google Classroom — Tarefa3.ipynb e Tarefa3.pptx.

4 Tarefa 4 — Exploração: Descritores Neurais com Kornia (DISK)

Por que explorar descritores neurais?

Nos últimos anos surgiram detectores e descritores baseados em redes neurais que superam os clássicos em cenários difíceis: mudanças drásticas de iluminação (dia/noite), diferentes estações do ano, pontos de vista muito distantes. Exemplos bem conhecidos são o **SuperPoint**, o **R2D2** e o **DISK** (*DIScrete Keypoints*). Todos aprendem, a partir de dados, o que é um bom keypoint — algo que os clássicos definem com regras matemáticas fixas.

Nesta tarefa, você **não** precisará entender como a rede é treinada. O objetivo é usar a biblioteca **Kornia** para rodar o **DISK** (modelo com boa integração em Kornia e tutorial oficial) em suas imagens e comparar os resultados com o SIFT clássico.

Aviso sobre versões

APIs de modelos pré-treinados em Kornia podem variar entre versões. Os trechos de código abaixo seguem a API documentada nos tutoriais oficiais e na documentação atual da Kornia. Se o seu `import` falhar, atualize com:

```
!pip install --upgrade kornia kornia-rs
```

E consulte a referência oficial em <https://kornia>.

[readthedocs.io/en/latest/feature.html](https://readthedocs.io/en/latest/feature.html).

Objetivos de Aprendizagem

- Instalar e usar Kornia para carregar o modelo DISK pré-treinado
- Detectar keypoints e extrair descritores neurais em um par de imagens
- Visualizar e comparar resultados com SIFT (quantidade, distribuição, matches)
- Refletir: em que situações esses modelos seriam mais vantajosos?

4.1 Material de Referência (sem teoria profunda)

- Slides da aula: slide 20 — *A Virada Neural* (R2D2, DISK, SuperPoint, LoFTR)
- **Torralba, A., Freeman, W. T., Isola, P.** (2024), *Foundations of Computer Vision*, MIT Press — Cap. 11, *Local Features and Matching*: ponte teórica formal entre os clássicos e os descritores neurais, ideal como leitura contextual antes desta tarefa
- Kornia — referência do módulo `feature`: <https://kornia.readthedocs.io/en/latest/feature.html>
- Tutorial oficial DISK: [https://www.kornia.org/tutorials/nbs/image\\_matching\\_disk.html](https://www.kornia.org/tutorials/nbs/image_matching_disk.html)
- Tutorial oficial LightGlue + DISK (bônus): [https://www.kornia.org/tutorials/nbs/image\\_matching\\_lightglue.html](https://www.kornia.org/tutorials/nbs/image_matching_lightglue.html)

Você **não** precisa ler artigos completos. Para o livro do Torralba, o Cap. 11 é acessível e serve como contexto sem exigir teoria profunda de redes.

4.2 Instalação e Configuração

```
# Google Colab
!pip install --upgrade kornia kornia-rs

import torch
import kornia
import kornia.feature as KF
from kornia.feature import DISK

print('Kornia:', kornia.__version__)
print('PyTorch:', torch.__version__)

device = torch.device(
    'cuda' if torch.cuda.is_available()
    else 'cpu')
print('Dispositivo:', device)
```

GPU no Google Colab

Ative GPU gratuita em: *Ambiente de execução* → *Alterar tipo de hardware* → *T4 GPU*. O DISK roda em CPU, porém mais devagar.

4.3 Ação — Exploração Prática

Use o mesmo par de imagens da Tarefa 2 ou da Tarefa 3.

Passo 1 — Carregar imagens como tensores:

```
import cv2
import torch

def load_tensor(path, device):
    img = cv2.imread(path)
    if img is None:
        raise FileNotFoundError(path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    t = torch.from_numpy(img).float() / 255.0
    return (t.permute(2, 0, 1)
            .unsqueeze(0)
            .to(device))

img1_t = load_tensor('esquerda.jpg', device)
img2_t = load_tensor('direita.jpg', device)
```

Passo 2 — Rodar o DISK:

```
# 'depth' e 'epipolar' sao os checkpoints
# disponiveis em DISK.from_pretrained.
disk = DISK.from_pretrained('depth').to(device)
disk.eval()

with torch.inference_mode():
```

```
feats1 = disk(
    img1_t, n=2048,
    pad_if_not_divisible=True)[0]
feats2 = disk(
    img2_t, n=2048,
    pad_if_not_divisible=True)[0]

kp1 = feats1.keypoints.cpu().numpy()
d1 = feats1.descriptors.cpu().numpy()
kp2 = feats2.keypoints.cpu().numpy()
d2 = feats2.descriptors.cpu().numpy()

print(f'DISK img1: {len(kp1)} kp ',
      f'| desc {d1.shape}')
print(f'DISK img2: {len(kp2)} kp ',
      f'| desc {d2.shape}')
```

**Passo 3 — Matching:** O DISK gera descritores *float*, então use NORM\_L2. Para descritores neurais o ratio test costuma exigir um limiar mais alto (0.85–0.95).

```
import cv2

bf = cv2.BFMatcher(cv2.NORM_L2)
pairs = bf.knnMatch(d1, d2, k=2)
good = []
for pair in pairs:
    if len(pair) == 2:
        m, n = pair
        if m.distance < 0.9 * n.distance:
            good.append(m)
print(f'DISK matches bons: {len(good)}')
```

**Bônus opcional — LightGlue:** a Kornia inclui um *matcher* neural que combina especialmente bem com DISK. Veja o tutorial oficial:

```
from kornia.feature import LightGlueMatcher
matcher = LightGlueMatcher('disk').to(device).eval()
# ver tutorial: image_matching_lightglue.html
```

**Passo 4 — Visualizar e comparar:** converta os kp1/kp2 do DISK para a estrutura cv2.KeyPoint para usar cv2.drawMatches() e compare lado a lado com o SIFT no mesmo par.

```
img1 = cv2.imread('esquerda.jpg')
img2 = cv2.imread('direita.jpg')

kp1_cv = [cv2.KeyPoint(float(x), float(y), 1)
           for x, y in kp1]
kp2_cv = [cv2.KeyPoint(float(x), float(y), 1)
           for x, y in kp2]

img_matches = cv2.drawMatches(
    img1, kp1_cv, img2, kp2_cv,
    good, None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
```

Tabela Comparativa Final

| Método | Tipo               | KP | Matches |
|--------|--------------------|----|---------|
| SIFT   | Clássico / float   | —  | —       |
| ORB    | Clássico / binário | —  | —       |
| AKAZE  | Clássico / binário | —  | —       |
| DISK   | Neural / float     | —  | —       |

- Reflexão final** (máximo 1 parágrafo por pergunta):
- Em que cenário o DISK foi visivelmente melhor que o SIFT? Em que cenário os resultados foram parecidos?
  - Quais são as desvantagens práticas de usar DISK em comparação com ORB em um sistema embarcado (câmera de robô, drone)?
  - Se você fosse escolher um único método para um sistema de vigilância *indoor* em tempo real, qual escolheria e por quê?

4.4 Critérios de Avaliação (25% cada)

- Contextualização:** Entendimento do que o DISK faz, sem exigir detalhes de treinamento.
  - Implementação:** Kornia funcionando; keypoints e descritores extraídos corretamente.
  - Comparação:** Tabela final preenchida; matching visualizado para o DISK e para o SIFT.
  - Reflexão Crítica:** As 3 perguntas respondidas com embasamento nos resultados observados.
- Submissão:** Google Classroom — Tarefa4.ipynb e Tarefa4.pptx.

Perguntas Frequentes (FAQ)

Dúvidas Comuns e Soluções Rápidas

Esta seção reúne respostas às dúvidas técnicas e conceituais mais frequentes durante a execução das tarefas no Google Colab com OpenCV e Kornia.

Problemas Técnicos

**Q:** “cv2.SIFT\_create() retorna erro”  
**R:** Atualize o OpenCV:

```
!pip install --upgrade opencv-python
```

Se ainda falhar, use a variante contrib:

```
!pip install --upgrade opencv-contrib-python
```

**Q:** “Kornia não encontra o modelo pré-treinado DISK”

**R:** Verifique a conexão com a internet no Colab — o download é automático na primeira execução. Atualize Kornia (!pip install -upgrade kornia). Se o erro for de assinatura da função (parâmetros mudaram entre versões), confirme a API atual no tutorial oficial: [https://www.kornia.org/tutorials/nbs/image\\_matching\\_disk.html](https://www.kornia.org/tutorials/nbs/image_matching_disk.html).

**Q:** “Meu panorama ficou com bordas pretas grandes ou imagens não alinhadas”

**R:** Verifique se as imagens foram fotografadas girando a câmera aproximadamente em torno do mesmo ponto, sem deslocar lateralmente o celular. Aumente o limiar de Lowe para 0.85. Se o problema persistir, redimensione para ~800 px de largura: cv2.resize(img, None, fx=0.5, fy=0.5).

**Q:** “Tenho poucos matches (<10) e a homografia falha”

**R:** Certifique-se de sobreposição de 30–50%. Para ORB, passe nfeatures=3000 ao criar o detector. Para SIFT, reduza contrastThreshold para 0.03. Aumente também o limiar de Lowe (0.75 → 0.85) para aceitar mais candidatos.

Dúvidas Conceituais

**Q:** “Qual a diferença entre detector e descritor?”

**R:** **Detector** → indica *onde* está o ponto (x, y, escala, orientação). **Descritor** → codifica *como* é o ponto (vetor numérico). O SIFT faz os dois; o Harris só detecta.

**Q:** “Por que NORM\_L2 para SIFT e NORM\_HAMMING para ORB?”

**R:** Descritores float representam magnitudes contínuas — a distância euclidiana (L2) mede o quão diferentes são dois vetores. Descritores binários são vetores de bits; a distância de Hamming conta quantos bits diferem e é calculada muito mais rapidamente com operações bit-a-bit.

**Q:** “O que é o Lowe’s Ratio Test e por que 0.75?”

**R:** Para cada keypoint encontramos os 2 melhores candidatos. Se o melhor for muito parecido com o segundo melhor (razão próxima de 1.0), o match é ambíguo e deve ser descartado. Lowe descobriu empiricamente que 0.75 elimina ~90% dos matches errados mantendo ~90% dos corretos. Para descritores neurais como o DISK, costuma-se usar limiares mais altos (0.85–0.95).

**Q:** “O que é o DISK e por que ele em vez de R2D2 ou SuperPoint?”

**R:** DISK é um modelo neural que aprende detector e descritor em conjunto, treinado com aprendizado por reforço para otimizar diretamente a qualidade do matching. Foi escolhido para a tarefa porque tem boa integração com Kornia e tutorial oficial. R2D2 e SuperPoint resolvem o mesmo problema com filosofias diferentes; em produção, a escolha depende do dataset, da plataforma e da licença.

**Q:** “Preciso de GPU para a Tarefa 4?”

**R:** Não obrigatoriamente. O DISK roda em CPU, porém mais devagar. No Google Colab, ative GPU gratuita em: *Ambiente de execução* → *Alterar tipo de hardware* → *T4 GPU*.